

TypeScript Advanced Classes & Inheritance

Learning Objectives

- Understand advanced class features in TypeScript
 - Master static properties and methods
 - Work with abstract classes and methods
 - Implement inheritance patterns effectively
 - Use accessor properties for controlled data access
-

TypeScript Classes - Advanced Features

Static Properties and Methods

Static members belong to the class itself, not to instances of the class. You can access them without creating an object. They are useful for utility functions or shared data across all instances.

```
class Calculator {
  static PI: number = 3.14159;

  static add(a: number, b: number): number {
    return a + b;
  }

  static circleArea(radius: number): number {
    return this.PI * radius * radius;
  }
}

// Access without creating instance
console.log(Calculator.PI);
console.log(Calculator.add(5, 3));
console.log(Calculator.circleArea(10));
```

Output:

3.14159

8

314.159

Key Points:

- Use `static` keyword before property or method name
- Access via class name, not instance name
- Inside static methods, `this` refers to the class itself
- Common use: configuration values, helper functions, counters

Abstract Classes and Methods

An abstract class is a blueprint for other classes. It cannot be instantiated directly. It can contain both complete methods and abstract methods (methods without implementation). Subclasses must implement all abstract methods.

```
abstract class Shape {
  constructor(public color: string) {}

  // Abstract method - no implementation
  abstract calculateArea(): number;

  // Concrete method - has implementation
  describe(): void {
    console.log(`A ${this.color} shape`);
  }
}

class Circle extends Shape {
  constructor(color: string, public radius: number) {
    super(color);
  }

  // Must implement abstract method
  calculateArea(): number {
    return 3.14 * this.radius * this.radius;
  }
}

class Square extends Shape {
  constructor(color: string, public side: number) {
    super(color);
  }

  calculateArea(): number {
    return this.side * this.side;
  }
}

const circle = new Circle("red", 5);
circle.describe();
console.log("Area:", circle.calculateArea());
```

```
const square = new Square("blue", 4);
square.describe();
console.log("Area:", square.calculateArea());
```

Output:

A red shape

Area: 78.5

A blue shape

Area: 16

Important Rules:

- Abstract classes use `abstract` keyword
- Cannot create instances of abstract classes directly
- Abstract methods have no body (no implementation)
- Child classes must implement all abstract methods
- Abstract classes can have regular methods with implementation

Class Expressions

Classes can be defined as expressions and assigned to variables. This is useful when you need to create classes dynamically or pass them as values.

```
// Named class expression
const User = class Person {
  constructor(public name: string) {}

  greet(): void {
    console.log(`Hello, I'm ${this.name}`);
  }
};

const user1 = new User("Alice");
user1.greet();

// Anonymous class expression
const Product = class {
  constructor(public name: string, public price: number) {}

  display(): void {
    console.log(`${this.name}: ${this.price}`);
  }
};
```

```
const product = new Product("Laptop", 999);
product.display();
```

Output:

```
Hello, I'm Alice
```

```
Laptop: $999
```

Accessor Properties (Getters and Setters)

Getters and setters allow you to control how properties are accessed and modified. They look like regular properties when used, but run functions behind the scenes.

```
class BankAccount {
  private _balance: number = 0;

  // Getter
  get balance(): number {
    console.log("Reading balance");
    return this._balance;
  }

  // Setter with validation
  set balance(amount: number) {
    if (amount < 0) {
      console.log("Balance cannot be negative!");
      return;
    }
    console.log("Updating balance");
    this._balance = amount;
  }

  deposit(amount: number): void {
    this.balance = this._balance + amount;
  }
}

const account = new BankAccount();
account.balance = 1000;    // Uses setter
console.log(account.balance); // Uses getter

account.balance = -500;    // Validation fails
console.log(account.balance);
```

Output:

```
Updating balance
Reading balance
1000
Balance cannot be negative!
Reading balance
1000
```

Benefits of Getters/Setters:

- Add validation logic when setting values
- Perform calculations when getting values
- Log access to properties
- Maintain backward compatibility when refactoring

Inheritance in TypeScript

What is Inheritance?

Inheritance is a mechanism where a new class (child class) is created from an existing class (parent class). The child class inherits properties and methods from the parent class and can add its own features or modify inherited ones.

Real-world analogy: Think of a generic "Vehicle" as a parent. A "Car" inherits common vehicle features (wheels, engine) but adds specific features (4 wheels, car doors).

Types of Inheritance

1. Single Inheritance

A child class extends one parent class. This is the most common form of inheritance.

```
class Animal {
  constructor(public name: string) {}

  makeSound(): void {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  constructor(name: string, public breed: string) {
    super(name); // Call parent constructor
  }
}
```

```

    bark(): void {
        console.log(`${this.name} barks: Woof!`);
    }
}

const dog = new Dog("Buddy", "Golden Retriever");
dog.makeSound(); // Inherited method
dog.bark();      // Own method
console.log(dog.breed);

```

Output:

```

Buddy makes a sound
Buddy barks: Woof!
Golden Retriever

```

2. Multilevel Inheritance

A class extends another class, which itself extends another class. This creates a chain of inheritance.

```

class LivingBeing {
    breathe(): void {
        console.log("Breathing...");
    }
}

class Animal extends LivingBeing {
    move(): void {
        console.log("Moving...");
    }
}

class Bird extends Animal {
    fly(): void {
        console.log("Flying...");
    }
}

const sparrow = new Bird();
sparrow.breathe(); // From LivingBeing
sparrow.move();    // From Animal
sparrow.fly();     // From Bird

```

Output:

Breathing...

Moving...

Flying...

3. Hierarchical Inheritance (Through Interfaces)

Multiple classes extend the same parent class. TypeScript also supports implementing multiple interfaces.

```
interface Flyable {  
  fly(): void;  
}  
  
interface Swimmable {  
  swim(): void;  
}  
  
class Bird implements Flyable {  
  fly(): void {  
    console.log("Bird is flying");  
  }  
}  
  
class Fish implements Swimmable {  
  swim(): void {  
    console.log("Fish is swimming");  
  }  
}  
  
class Duck implements Flyable, Swimmable {  
  fly(): void {  
    console.log("Duck is flying");  
  }  
  
  swim(): void {  
    console.log("Duck is swimming");  
  }  
}  
  
const duck = new Duck();  
duck.fly();  
duck.swim();
```

Output:

Duck is flying

Duck is swimming

The extends Keyword

The `extends` keyword creates an inheritance relationship between classes.

```
class Employee {
  constructor(public name: string, public id: number) {}

  work(): void {
    console.log(`${this.name} is working`);
  }
}

class Manager extends Employee {
  constructor(name: string, id: number, public team: string[]) {
    super(name, id);
  }

  assignTask(task: string): void {
    console.log(`${this.name} assigned: ${task}`);
  }
}

const manager = new Manager("Sarah", 101, ["Alice", "Bob"]);
manager.work();
manager.assignTask("Code review");
```

Output:

```
Sarah is working
```

```
Sarah assigned: Code review
```

Method Overriding

A child class can provide a different implementation for a method inherited from the parent class.

```
class Printer {
  print(document: string): void {
    console.log(`Printing: ${document}`);
  }
}

class ColorPrinter extends Printer {
  // Override parent method
  print(document: string): void {
    console.log(`Printing in COLOR: ${document}`);
  }
}

const basic = new Printer();
```



```
basic.print("Report.pdf");

const color = new ColorPrinter();
color.print("Report.pdf");
```

Output:

```
Printing: Report.pdf
Printing in COLOR: Report.pdf
```

Using super Keyword

The `super` keyword refers to the parent class. It is used to call the parent constructor and parent methods.

```
class Vehicle {
  constructor(public brand: string) {
    console.log("Vehicle constructor called");
  }

  start(): void {
    console.log("Vehicle starting...");
  }
}

class Car extends Vehicle {
  constructor(brand: string, public model: string) {
    super(brand); // Must call parent constructor
    console.log("Car constructor called");
  }

  start(): void {
    super.start(); // Call parent method
    console.log(`${this.brand} ${this.model} engine started`);
  }
}

const car = new Car("Toyota", "Camry");
car.start();
```

Output:

```
Vehicle constructor called
Car constructor called
Vehicle starting...
Toyota Camry engine started
```

Key Rules for super:

- `super()` must be called in child constructor before using `this`
- `super.methodName()` calls parent's version of method
- Useful when you want to extend parent behavior, not replace it

Practice Exercise

Create an abstract class `Payment` with an abstract method `processPayment(amount: number)` and a concrete method `printReceipt()`. Then create two classes `CreditCard` and `Cash` that extend `Payment` and implement their own payment processing logic.

```
abstract class Payment {
  constructor(public transactionId: string) {}

  abstract processPayment(amount: number): void;

  printReceipt(): void {
    console.log(`Receipt - Transaction ID: ${this.transactionId}`);
  }
}

class CreditCard extends Payment {
  constructor(transactionId: string, public cardNumber: string) {
    super(transactionId);
  }

  processPayment(amount: number): void {
    console.log(`Processing ${amount} via card ${this.cardNumber}`);
  }
}

class Cash extends Payment {
  processPayment(amount: number): void {
    console.log(`Received ${amount} in cash`);
  }
}

const card = new CreditCard("TXN001", "**** 1234");
card.processPayment(150);
card.printReceipt();

const cash = new Cash("TXN002");
cash.processPayment(50);
cash.printReceipt();
```

Output:

Processing \$150 via card **** 1234

Receipt - Transaction ID: TXN001

Received \$50 in cash

Receipt - Transaction ID: TXN002

Summary

In this module, you learned:

- Static properties and methods for class-level functionality
- Abstract classes as blueprints for other classes
- Class expressions for dynamic class creation
- Getters and setters for controlled property access
- Inheritance types: single, multilevel, and hierarchical
- Method overriding and the super keyword

These concepts form the foundation for building scalable and maintainable TypeScript applications. Practice these patterns to become comfortable with object-oriented programming in TypeScript.